Lane to Byte Mapping Functional Design Specifications

UCIe 3.0 Physical Layer

Version 1

Block Owner

ENG. Karim Waseem

Authors

Mahmoud Hesham

This Page is left Blank Intentionally

1 Table of Contents

2 Revision History

Version	Date	Author(s)	Revision Notes	Owner Approval

3 Overview

This block implements a **configurable multi-lane data aggregation and packing unit**.
It receives **16 parallel input lanes**, each **64 bits wide**, and assembles them into a **single 2048-bit output word**.

The block supports multiple **lane mapping modes** (x16, x8, x4) selected via a lane map control code. Based on the selected mode, input lanes are conditionally enabled, multiplexed, and accumulated using **mode-specific shift registers**. A final output multiplexer selects the correctly packed data path and drives the output interface.

4 Operation and Description

4.1 Digital Interface

4.1.1 Parameters Names

Parameter Name	Default	Description
pDATA_IN_WIDTH	64	Input data word width
pDATA_OUT_WIDTH	2048	Output data word width

4.1.2 Ports Names

Port Name	Port Width	Port Type	Description
i_lane_0	pDATA_IN_WIDTH	input	Parallel input data lanes that will be packed. Each lane provides a 64-bit data word at each cycle
i_lane_1		input	
i_lane_2		input	
i_lane_3		input	
i_lane_4		input	
i_lane_5		input	
i_lane_6		input	
i_lane_7		input	
i_lane_8		input	
i_lane_9		input	
i_lane_10		input	
i_lane_11		input	
i_lane_12		input	
i_lane_13		input	
i_lane_14		Input	
i_lane_15		input	
i_lane_map_code [2:0]	3	input	Lane Map Code. Selects lane mapping configuration as listed in 4.1.3.
i_clk	1	input	Clock used in design
i_enable	1	input	Enable for lane to byte mapping block
i_reset	1	input	Reset for the system
i_lane_map_code	3	input	Control signals determine functional lanes
o_pl_valid	1	input	Flag to controllers that indicates data packed and valid
o_pl_data	pDATA_OUT_WIDTH	output	Output packed data

4.1.3 Standard Package Logical Lane Map

Lane Map Code	**Functional Lanes**	**Decodings{mux_8,mux_4,x16_en,x8_en,x4_en}**
000b	None (Degrade not possible)	0 0 1 0 0
001b	Logical Lanes 0 to 7	0 0 0 1 0
010b	Logical Lanes 8 to 15	1 0 0 1 0
011b	0 - 15	0 0 1 0 0
100b	Logical Lanes 0 to 3	0 0 0 0 1
101b	Logical Lanes 4 to 7	0 1 0 0 1

mux_8 : determines which lanes will be taken (0 to 7) or (8 to 15)

mux_4: determines which lanes will be taken (0 to 3) or (4 to7)

en_x16 : enable signal for x16 lanes shift registers

en_x8 : enable signal for x8 lanes shift registers

en_x4 : enable signal for x4 lanes shift registers

4.2 Functional Description

4.2.1 Input Interface and Control Signals

- Sixteen parallel input data lanes (i_lane_0 to i_lane_15) provide 64-bit data words.
- All inputs are sampled on the rising edge of i_clk when i_enable is asserted.
- The i_reset signal clears all internal storage elements and control logic.

4.2.2 Lane Mapping Decoder

- The i_lane_map_code input is decoded to generate mode-specific enable signals:
  - x16_en , x8_en , x4_en
  - mux_x4 , mux_x8
- These enable signals control:
  - Which shift registers are active
  - Which multiplexers are enabled
  - The final output selection

Only one lane mapping mode is active at any given time.

4.2.3 Enable Gating Logic

- Each mapping mode enable is logically ANDed with i_enable.
- This ensures that no data is captured or shifted unless the block is explicitly enabled.

4.2.4 Data Multiplexing and Grouping

Depending on the selected mode, input lanes are grouped using multiplexers:

x16 Mode

- Each input lane feeds its own **128-bit shift register**.
- All 16 lanes operate independently and in parallel.

x8 Mode

- Input lanes are selected using 2-to-1 multiplexers.
- Each pair feeds a **256-bit shift register**.
- Eight shift registers operate in parallel.

x4 Mode

- Input lanes are selected using 2-to-1 multiplexers.
- Each group feeds a **512-bit shift register**.

- Four such shift registers operate in parallel.

The multiplexers ensure correct lane selection and alignment for each mode.

4.2.5 Shift Register Accumulation

- Shift registers accumulate incoming 64-bit data words on each clock cycle.
- Data is shifted until the full register width is filled.
- The number of clock cycles required to complete accumulation depends on the active lane mapping mode.
- All shift registers are synchronously reset by i_reset.

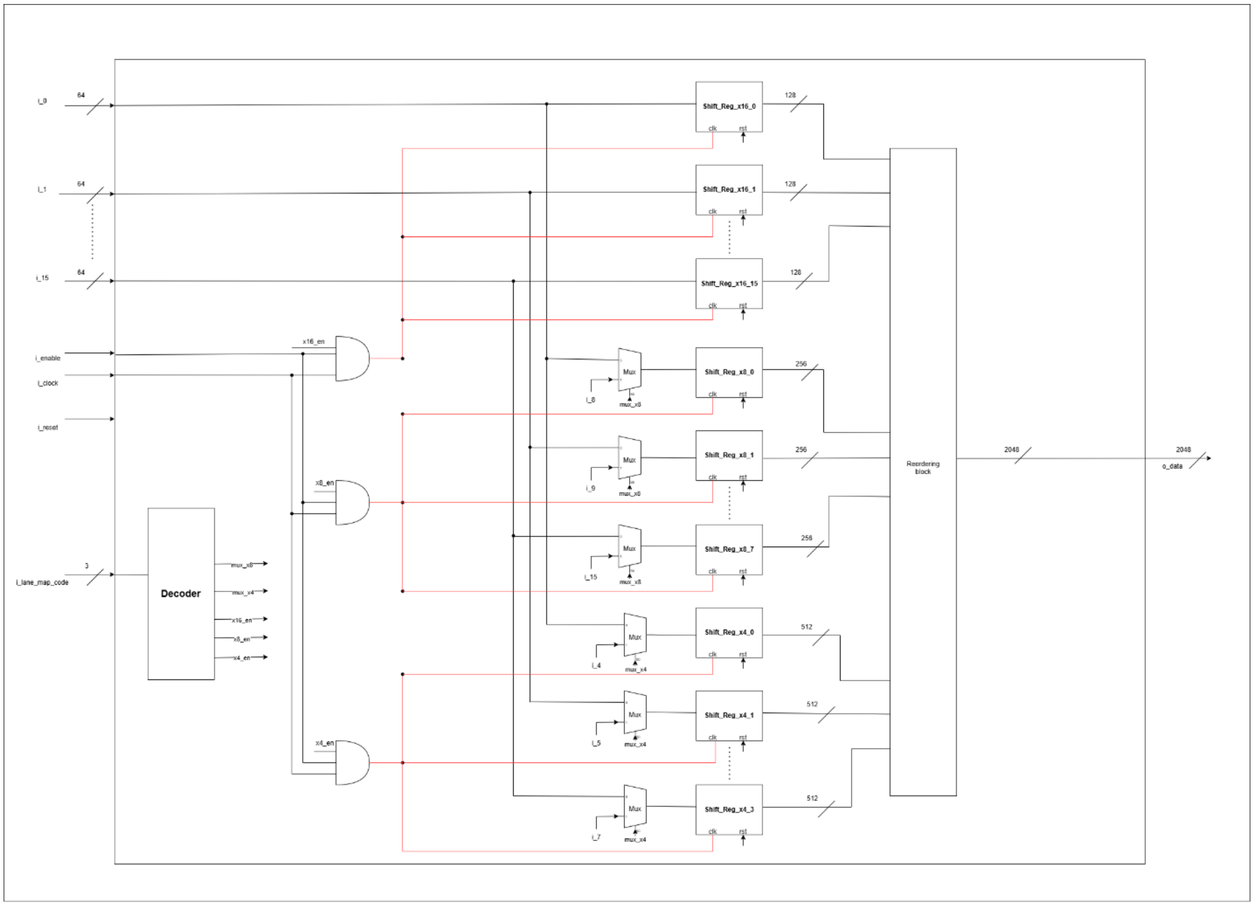
4.2.5 Reordering block

- Concatenates output data from shift register in its order
- It works in 3 modes x16 , x8 and x4

4.2.6 Output Selection and Formatting

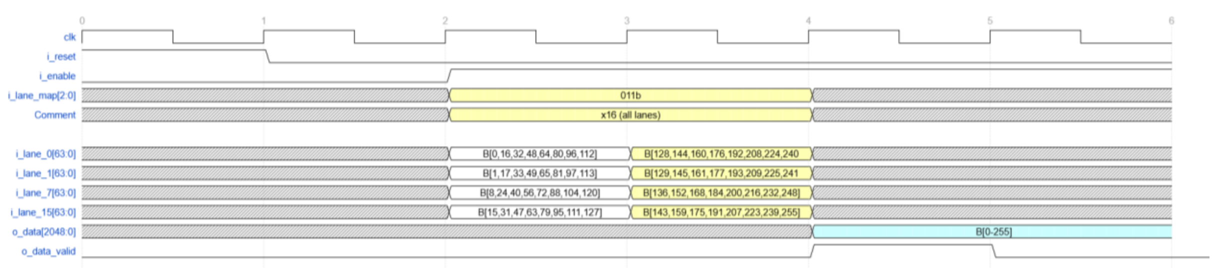
- Outputs of the active shift register bank are routed to a **final wide multiplexer**.
- The multiplexer selects the correct 2048-bit packed data based on the active lane mapping mode.
- The selected data is driven on the o_data output bus.

4.3 Block Diagram



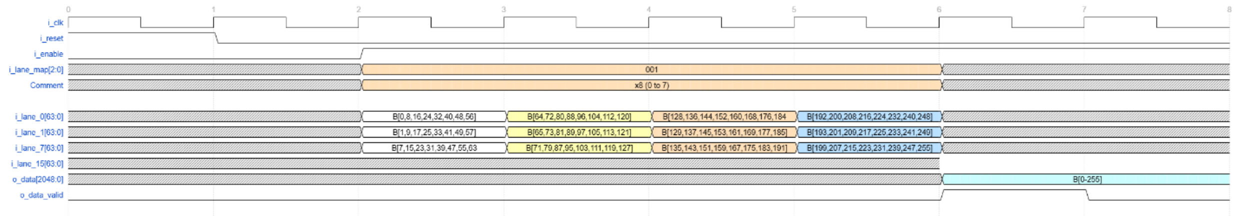
4.4 Timing Diagram

<u>x16 lane mode:</u>

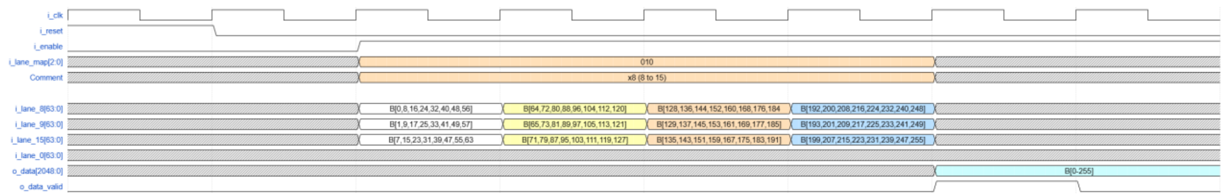


<u>x8 Lane mode:</u>

Lanes (0 to 7):

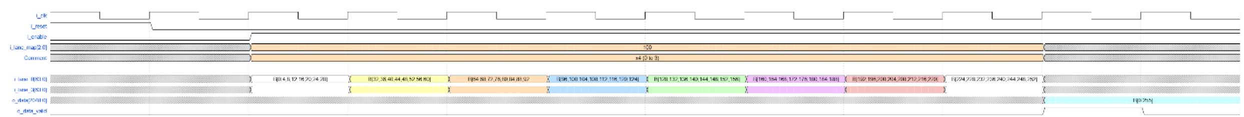


Lanes(7 to 15):

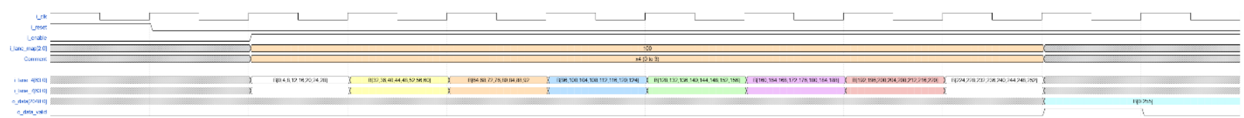


<u>x4 Lane mode:</u>

Lanes (0 to 3):



Lanes(4 to 7):



4.5 Verification Requirements

4.5 Verification Requirements

The following verification scenarios shall be implemented to ensure correct functionality:

4.5.1 Reset Verification

- Assert i_reset and verify that:
    - o All shift registers are cleared
    - o o_data is driven to zero or an invalid state
- Deassert reset and confirm normal operation resumes.

4.5.2 Enable Control Verification

- Verify that data shifting occurs only when i_enable is asserted.
- Confirm that deasserting i_enable freezes internal state.

4.5.3 Lane Mapping Mode Verification

- Independently verify each lane mapping mode:
  - x16 mode
  - x8 mode
  - x4 mode
- Confirm correct enable activation and correct mux selection per mode.

4.5.4 Data Packing and Ordering

- Apply known data patterns on input lanes.
- Verify correct lane ordering, alignment, and bit placement within o_data.
- Confirm valid data flag asserted when data is ready.